

Java EE

SERVLET ET VUE

Contenu

- Architecture
- Présentation du webcontainer
- Servlet
- Servlet par l'exemple

Qu'est ce qu'une Servlet ?

- Les servlets Java :
 - Une technologie pour générer des pages web dynamiques (comme PHP, ASP, ASP.NET, ...)
 - Tournent dans des serveurs dédiés dits « serveurs de servlets »,
 - Possèdent un contexte qui permet de communiquer avec la configuration du serveur
- La classe `HttpServlet`
 - Permet la génération de contenu web dynamique (HTML, XML, JSON, etc ...)

Qu'est ce qu'une Servlet ? (2)

- Servlets
 - Fournissent donc un framework général pour des services basés sur le paradigme requête-réponse
 - Portables sur n'importe quel serveur HTTP Java (léger comme Tomcat, Resin, ou plus complet comme Jboss, Glassfish, Websphere)
 - Ont accès à la famille des APIs de Java EE
 - JDBC, JPA, EJB, JMS, JAX-WS, JTA, JTS, RMI, JNDI, JAXP, ...
 - Sont des containers pour l'injection de code (CDI)

Servlet Services

- Java Servlets fournissent de nombreux “services”
 - API de bas niveau pour construire des services internet (requête, réponse, session, cookies, etc.)
 - Element fondamental derrière les Java Server Pages (JSP) et Java Server Faces (JSF)
 - Peuvent délivrer plusieurs types de réponses
 - XML, HTML, WML, GIF, etc...
 - Servent de “Contrôleur web” dans les architectures MVC JSP/Servlets (obsolète)

Pourquoi utiliser des Servlets ?

- Java
 - portable et répandu... c'est au coeur de la partie HTTP de Java EE
- Puissance
 - Utilisation de toutes les APIs Java,
 - Puissant mécanisme d'annotations,
 - Intégration dans plusieurs profils "Java EE" : profil léger Web, gros profil "Entreprise" pour faire des applications en clusters, etc
- Efficace
 - Highly scalable, code ré-entrant, compilé
 - Jusqu'à 100x plus rapide que PHP, même que C++

Pourquoi utiliser des Servlets (2)

- Sécurité
 - Typage fort,
 - Gestion de la mémoire efficace
- Intégration forte avec le serveur
 - Via des variables “contexte” échanges forts entre le serveur et les servlets : configuration, partage de ressources (connexions BD) etc.
- Extensibilité & Flexibilité
 - Théoriquement, le modèle Servlet n'est pas que pour HTTP,
 - Puissants mécanismes de “filtres” et de “chainage” de traitements. Plus de détails plus tard.

Date Servlet – Exemple

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DateServlet extends HttpServlet {
    public void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        // A éviter, normalement pas de HTML dans une servlet
        PrintWriter out = Res response.getWriter();
        out.println("<HTML>");
        out.println("The time is: " +
            new java.util.Date());
        out.println("</HTML>");
    }
}
```

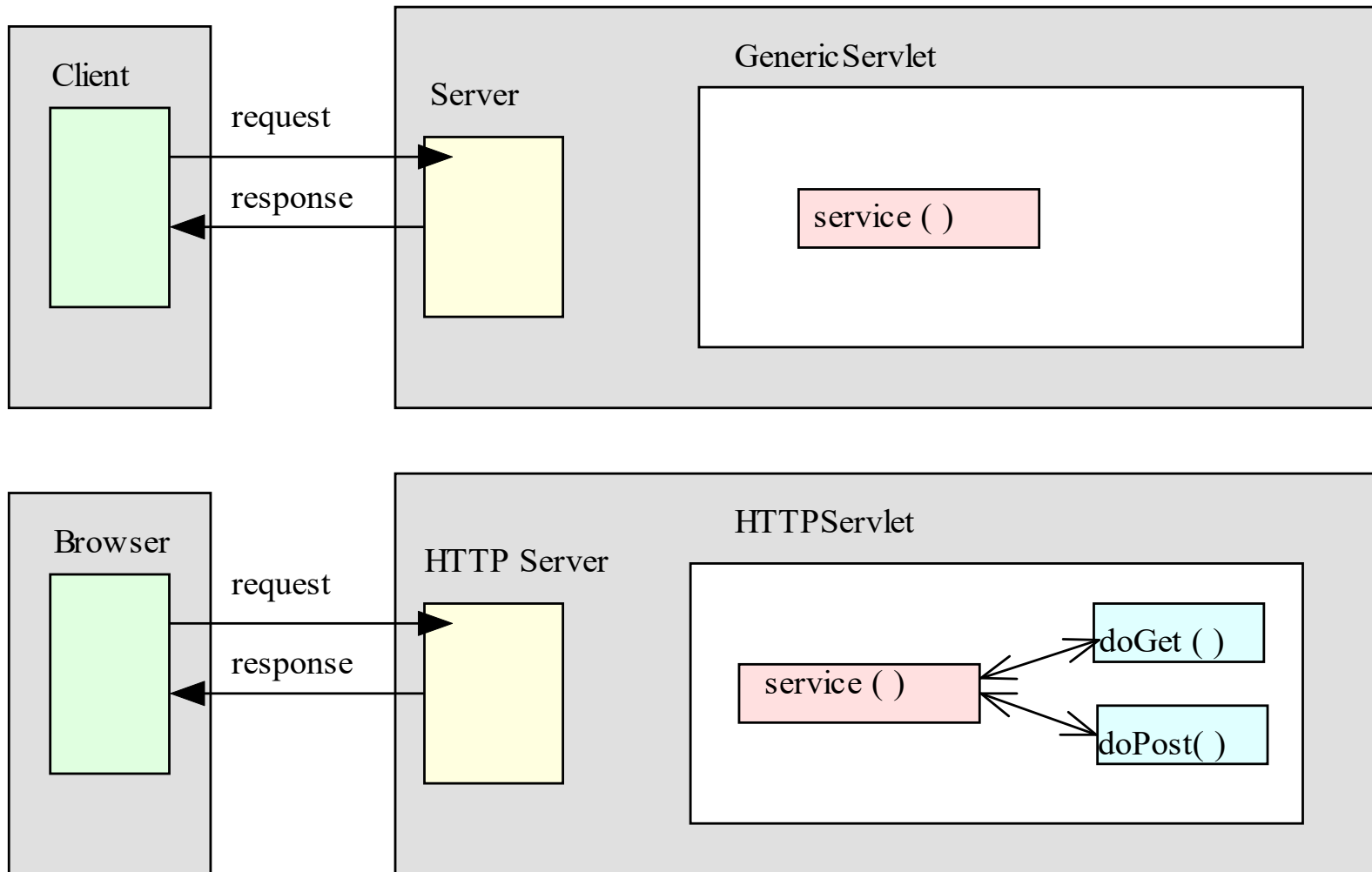
Java Servlets

ARCHITECTURE TECHNIQUE

Architecture d'une Servlet

- La classe HttpServlet
 - Rôle = traiter la requête HTTP
- Pour chaque méthode HTTP :GET, POST, PUT, DELETE, etc. il y a une méthode correspondante :
 - doGet(...) – répond aux requêtes HTTP GET
 - doPost(...) – répond aux requêtes HTTP GET
 - doPut(...), doHead(...), doDelete(...), doTrace(...), doOptions(...)
- Conseil : implémenter au moins les deux premières méthodes ou redéfinir la méthode service(...)

Servlet HTTP et servlet générique



Architecture d'une Servlet (2)

- Ces méthodes ont deux paramètres : la requête et la réponse
- L'objet `HttpServletRequest`
 - Contient la requête du client
 - En-tête de la requête HTTP
 - Paramètres HTTP (données de formulaire ou paramètres passés avec l'URL)
 - Autres données (cookies, URL, chemin relatif, etc.)
- L'objet `HttpServletResponse`
 - Encapsule les données renvoyées au client
 - En-tête de réponse HTTP (content type, cookies, etc.)
 - Corps de la réponse (en tant que `OutputStream`)

Architecture d'une Servlet (3)

- La méthode HTTP GET est utilisée quand :
 - Le traitement de la requête ne modifie pas l'état du serveur,
 - Si jamais il s'agit de l'envoi d'un formulaire, taille des données < 255 caractères,
 - On veut pouvoir bookmarker l'URL
- La méthode HTTP POST est utilisée quand :
 - Le traitement change l'état du serveur : exemple classique = un insert dans une base de données.
 - La quantité de données à envoyer est importante,
 - On veut cacher dans l'URL les paramètres (ex : mot de passe)

Servlets API

- Fonctions principales d'une Servlet :
 - Traiter les paramètres HTTP reçus dans la requête (GET ou POST)

```
HttpServletRequest.getParameter(String)
```

- Récupérer un paramètre de configuration de l'application web (dans le descripteur web.xml)

```
ServletConfig.getInitParameter()
```

- Récupérer un élément de l'en-tête HTTP

```
HttpServletRequest.getHeader(String)
```

Servlets API (2)

- Spécifier dans le header de réponse le type

```
HttpServletResponse.setHeader(<name>, <value>) /  
HttpServletResponse.setContentType(String)
```

- Récupérer un Writer pour écrire la réponse

```
HttpServletResponse.getWriter()
```

- ...ou un flux binaire si la réponse est binaire

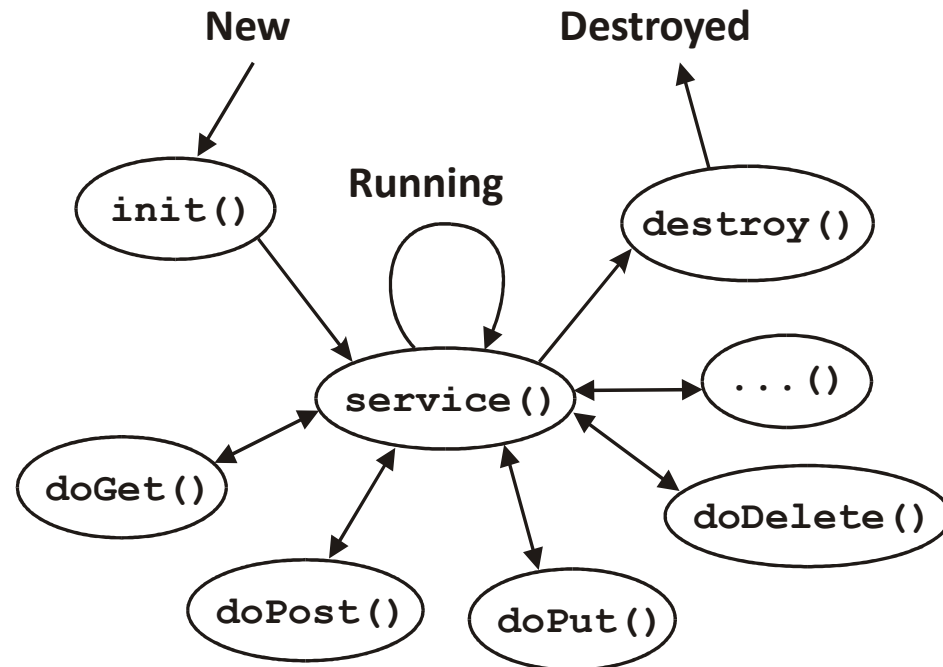
```
HttpServletResponse.getOutputStream()
```

- Rediriger la requête vers un autre URL

```
HttpServletResponse.sendRedirect()
```

Cycle de vie d'une Servlet

- ✓ Le serveur gère ce cycle,
- ✓ Les méthodes “callback” du cycle de vie ne doivent jamais être appelées par le code qu'on écrit



- On implémente souvent la méthode `init` (invoquée lors de la première exécution de la servlet) pour récupérer des ressources ou paramètres d'initialisation.

La méthode init()

- Appelée à la première instanciation,
- La spécification garantit qu'aucune requête ne sera envoyée avant que init() soit exécutée entièrement.
- On redéfinit init() quand :
 - On doit ouvrir des ressources (connexions JDBC par ex)
 - On doit initialiser l'état d'une Servlet,

La méthode service()

- Implémentée dans HttpServlet dont on hérite,
- Détecte la méthode HTTP et appelle doGet(...), doPost(...), Sends the result as HTTP response
- On ne la redéfinit pas dans les cas classiques
 - Elle est redéfinie par les librairies d'implémentation des web services, par JSF etc...

La méthode destroy()

- Appelée avant que la Servlet soit garbage collectée,
- La spécification garantit que toutes les requêtes seront traitées avant l'appel à destroy()
- On la redéfinit lorsque :
 - On doit libérer des ressources,
 - On doit rendre persistant l'état de la servlet (en général fait par des librairies spécialisées comme les Web Services).

Java Servlets

EXAMPLE

Exemple de traitement de paramètre : Hello Servlet

- Servlet qui récupère le paramètre "nom" et qui répond : "Hello, <nom>"

- Formulaire HTML :

```
<form method="GET ou POST" action="URL relatif de la Servlet">  
  <input type="text" name="nom">  
</form>
```

- Récupération du paramètre par la méthode doGet() ou doPost() :

```
String nom= request.getParameter("nom") ;
```

Hello Servlet : exemple complet

HelloForm.html

```
<html><body>
<form method="GET" action="HelloServlet">
  Entrez votre nom :
  <input type="text" name="nom">
  <input type="submit" value="OK">
</form>
</body></html>
```

HelloServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {
```

Hello Servlet – Example

HelloServlet.java

```
public void doGet(HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {

    response.setContentType("text/html");
    ServletOutputStream out = response.getOutputStream();
    String nom= request.getParameter("nom");
    out.println("<html><head>");
    out.println("\t<title>Hello Servlet</title>");
    out.println("</head><body>");
    out.println("\t<h1>Hello, " + nom + "</h1>");
    out.println("</body></html>");

}
```

Hello Servlet – requête HTTP

- What happens when the user enters his name?
 - Internet Explorer (IE) sends the following HTTP request to Tomcat

```
GET /FirstWebApp/HelloServlet?user_name=Nakov HTTP/1.1
Accept: image/gif, image/x-bitmap, image/jpeg,image/pjpeg,
application/vnd.ms-excel, application/vnd.ms-powerpoint,
application/msword, application/x-shockwave-flash, */*
Accept-Language: bg
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT
5.1; Q312461)
Host: nakov:8084
Connection: Keep-Alive
```

Servlet qui construit et sert une image

- On veut faire une Servlet qui construit une image avec un texte dedans : un compteur de vues.
- La Servlet maintient un compteur
 - On l'initialise dans la méthode `init()` et on l'incrmente dans la méthode `doGet()`
- La Servlet produit une image binaire
 - Le Content-type doit être "image/jpeg"

Image Counter Servlet (1)

```
import javax.servlet.*;
import javax.servlet.http.*;
...

public class ImageCounterServlet extends HttpServlet {
    private String mStartDate;
    private int mVisitCounter;

    public void init() {
        mStartDate = (new Date()).toString();
        mVisitCounter = 0;
    }

    public BufferedImage createImage(String msg) {
        ... // ici on crée une image, on dessine un texte
            // dedans et on renvoie l'image sous forme de
            // BufferedImage (buffer binaire)
    }
}
```

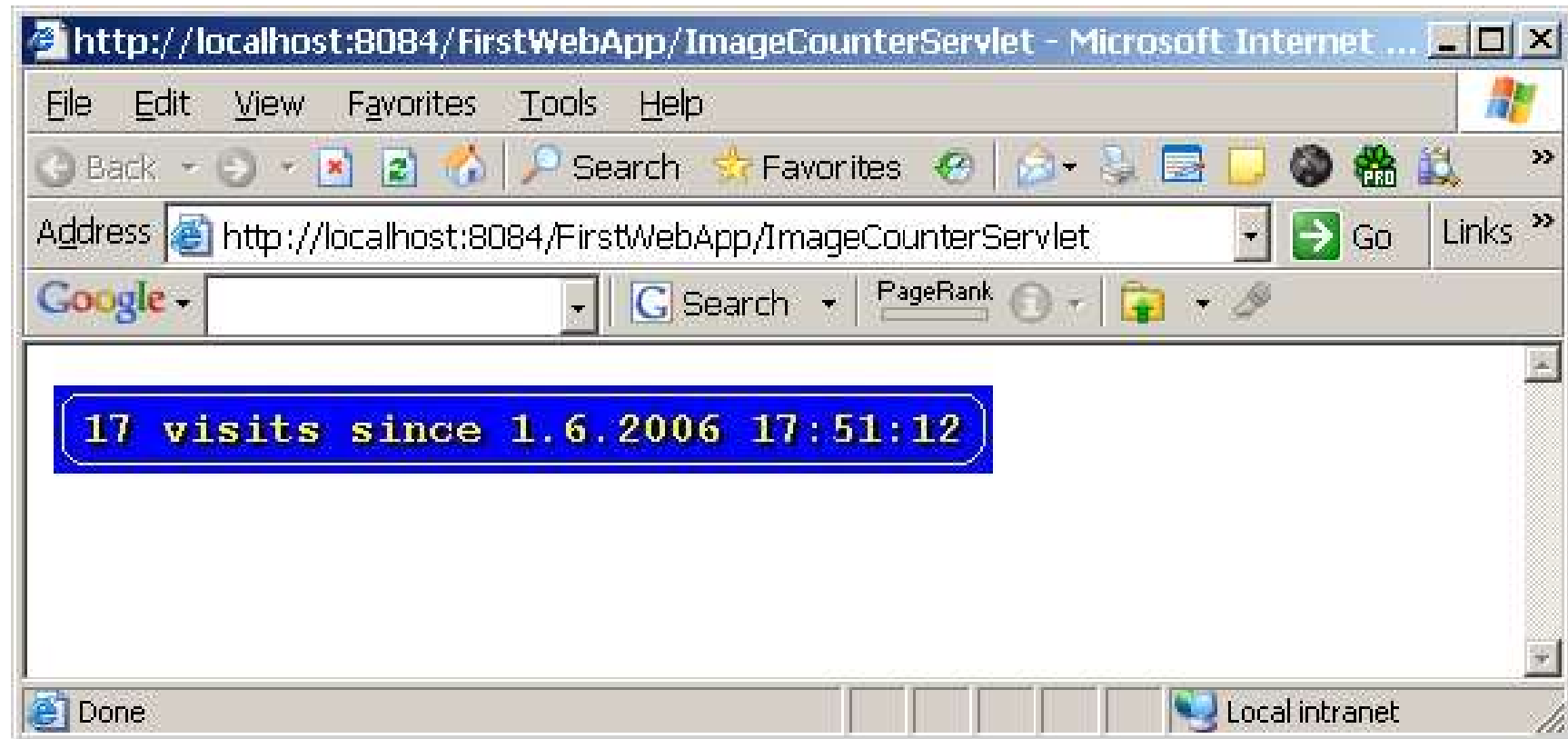
Image Counter Servlet (2)

```
public void doGet(HttpServletRequest request,
    HttpServletResponse response)
    throws IOException, ServletException {
    String msg;
    // Rappel les servlets sont ré-entrantes !
    synchronized(this) {
        mVisitCounter++;
        msg = "" + mVisitCounter + " visits since" +
            mStartDate;
    }
    BufferedImage image = createImage(msg);
    response.setContentType("image/jpeg");
    OutputStream out = response.getOutputStream();
    // Encode l'image en jpeg et l'écrit sur le flux
    // binaire de la réponse
    ImageIO.write(image, "jpg", out);
}
}
```

Extrait du code de création de l'image

```
// Create an image of our counter to be sent to the browser.  
  
BufferedImage buffer = new BufferedImage(50, 20, BufferedImage.TYPE_INT_RGB);  
  
Graphics2D g = buffer.createGraphics();  
  
g.setRenderingHint(RenderingHints.KEY_ANTIALIASING,  
                    RenderingHints.VALUE_ANTIALIAS_ON);  
  
g.setFont(new Font("Monospaced", Font.PLAIN, 14));  
  
g.setColor(Color.BLUE);  
  
g.fillRect(0, 0, 50, 20);  
  
g.setColor(Color.YELLOW);  
  
g.drawString(msg, 0, 20);  
  
return buffer;
```

Résultat



Renvoyer un zip depuis une Servlet

- Exemple de <http://www.kodejava.org/examples/590.html>
- ```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
 try {
 String path = getServletContext().getRealPath("data");
 File directory = new File(path);
 String[] files = directory.list();
 if (files != null && files.length > 0) {
 byte[] zip = zipFiles(directory, files);
 ServletOutputStream sos = response.getOutputStream();
 response.setContentType("application/zip");
 response.setHeader("Content-Disposition", "attachment;
 filename=\"DATA.ZIP\"");
 sos.write(zip); sos.flush();
 }
 } catch (Exception e) {
 e.printStackTrace();
 }
}
```

# Récupérer l'URL relatif de la servlet

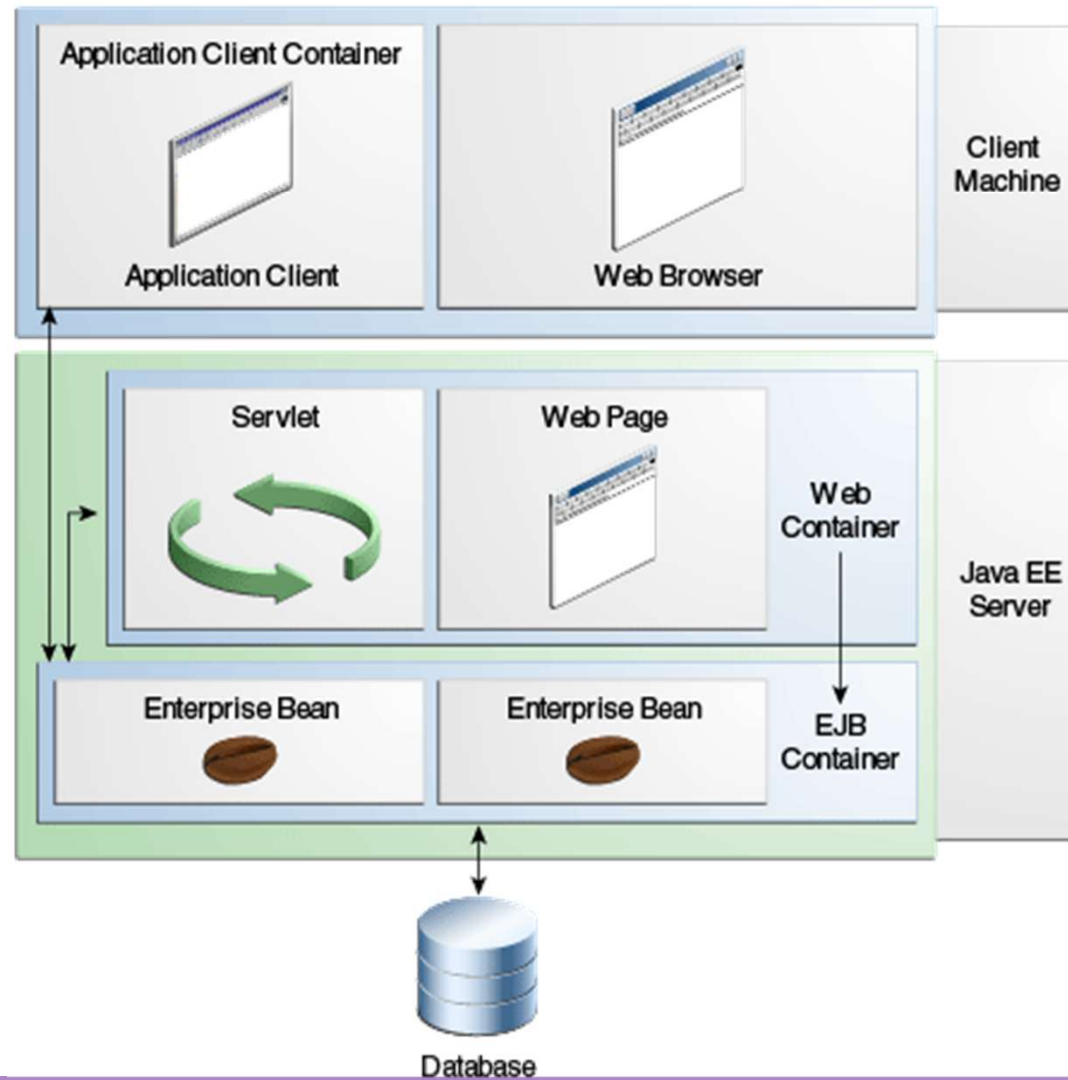
---

- Permet d'obtenir des informations depuis le header de la requête. Ex : chemin relatif utilisé pour appeler la servlet :
- ```
public class ContextPathDemo extends HttpServlet {  
    protected void doGet(HttpServletRequest req,  
        HttpServletResponse res) throws ServletException,  
        IOException {  
        // HttpServletRequest.getContextPath() returns the  
        // portion of the request URI that indicates the  
        // context of the request.  
        String contextPath = req.getContextPath();  
    }  
}
```

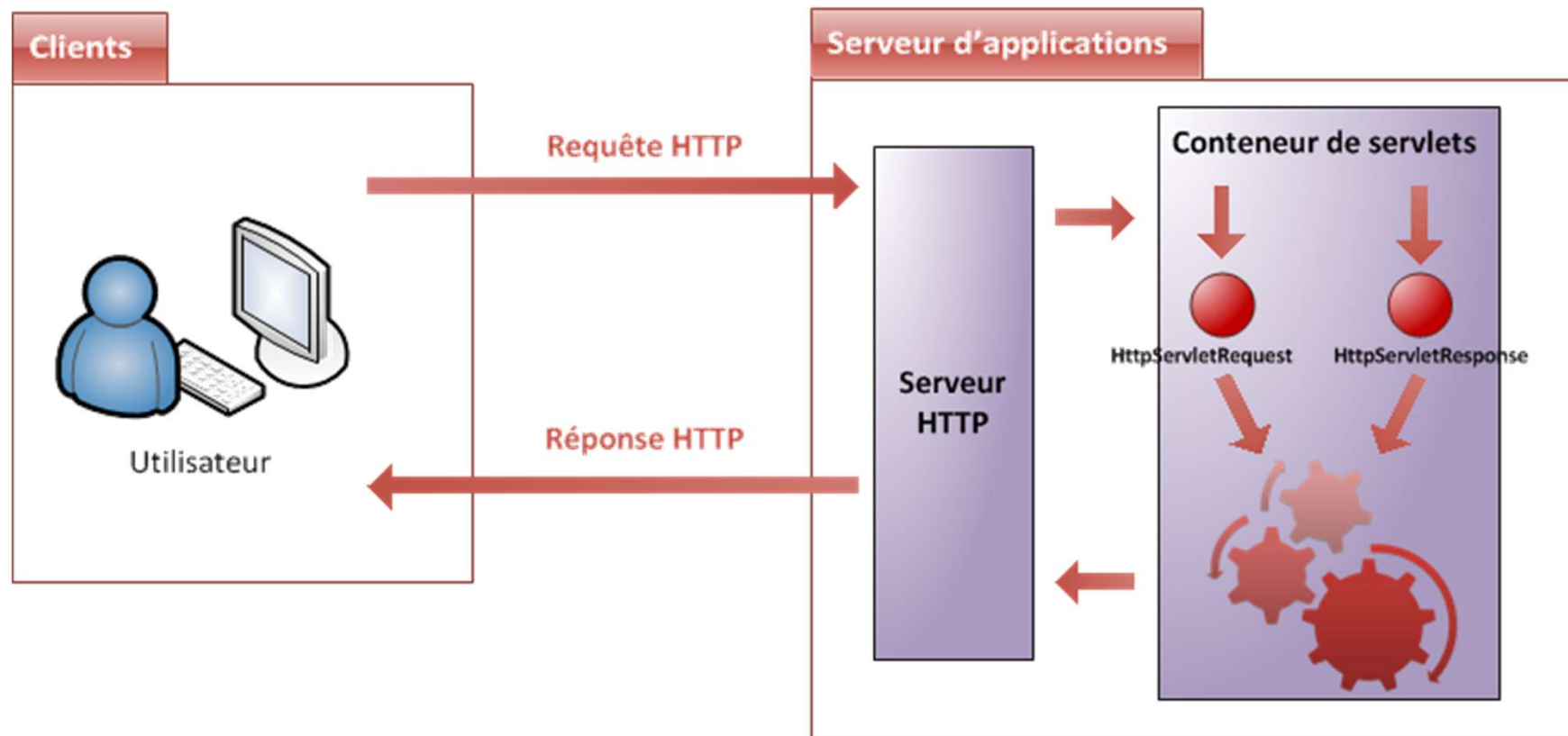
URI relatif à quoi cela sert-il ?

- Dans le cas où une même servlet est mappé sur plusieurs URLs,
- Exemple, une Servlet qui streame des fichiers mp3 dans /mp3/* a besoin de récupérer le nom des fichiers... /mp3/toto.mp3, /mp3/titi.mp3, on a besoin de récupérer toto.mp3 pour l'ouvrir...

Architecture



Fonctionnement



Classe Servlet

- Interface de base de toutes les servlets
- Classes implémentant Servlet :
 - GenericServlet
 - HttpServlet
 - Abstraite
 - 3 méthodes :
 - doGet()
 - doPost()
 - doHead()

Suite du tutoriel : création d'une servlet

- Nouvelle classe
 - Hérite de HttpServlet
 - Dans un package Servlet

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<web-app
```

```
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
  xmlns="http://xmlns.jcp.org/xml/ns/javaee"
```

```
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee  
    http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
```

```
  id="WebApp_ID"
```

```
  version="3.1">
```

```
</web-app>
```

Identification de la servlet

```
<servlet>  
  <servlet-name>Test</servlet-name>  
  <servlet-class>Servlet.Test</servlet-class>  
</servlet>
```

Association de la servlet à un url

```
<servlet-mapping>  
  <servlet-name>Test</servlet-name>  
  <url-pattern>/UrlTest</url-pattern>  
</servlet-mapping>
```

http://localhost:8080/Test/UrlTest

- Erreur 405
- Implémenter la méthode
 - **public void doGet(HttpServletRequest request, HttpServletResponse response)**